Implementation of FastSpeech 2-Based Text-to-Speech Synthesis for English and Telugu

Technical Report

Submitted by

Savio Shaju

Department of Computer Science and Engineering

Saintgits College of Engineering

GitHub Repository

https://github.com/savioshaju/TTS.git

Model Checkpoints and Audio Artifacts (Google Drive)

https://drive.google.com/drive/folders/1MYS5zcsyehxDN5n4XaKj5jCidpUZ-efw?usp=sharing

Abstract

This technical report documents the implementation, training, and evaluation of two independent FastSpeech 2 non-autoregressive text-to-speech (TTS) synthesis systems for Indian English and Telugu. Designed to overcome the exposure bias and slow inference of autoregressive architectures, FastSpeech 2 predicts mel-spectrograms non-autoregressively conditioned on phoneme or character embeddings and multi-task variance predictors for duration, pitch (F_0), and energy. Both systems were developed under an assignment data constraint of approximately 5 hours of training audio and 1 hour of validation audio. Indian English utilizes crowdsourced multi-speaker recordings from the AI4Bharat Svarah corpus aligned with the Montreal Forced Aligner (MFA) across 100 IPA phoneme tokens. Telugu utilizes studio recordings from the IndicTTS corpus aligned directly with Meta's Massively Multilingual Speech (MMS) Wav2Vec2-CTC acoustic model over 460 native-script akshara units. Both acoustic models were trained for 100,000 steps with mixed precision on dual NVIDIA Tesla T4 GPUs and paired with a pretrained HiFi-GAN vocoder adapted via sinc-resampling and duration calibration. Systematic evaluation on five prescribed benchmark test sentences per language demonstrates robust phoneme articulation, natural prosodic contour variation, and resilience against typographical anomalies and loanwords.

Contents

Abstract **i**

1 Introduction **1**
1.1 Background and Problem Statement . . . 1
1.2 FastSpeech 2 Architecture Principles . . . 1
1.3 Summary of Key Experimental Results . . . 2
1.4 Report Organization . . . 2

2 Implementation Details **3**
2.1 Base Architecture and Dependencies . . . 3
2.2 Codebase Structure . . . 3
2.3 Codebase Corrections and Bug Fixes . . . 3
2.4 Language-Specific Modifications . . . 4

3 Datasets and Preprocessing **5**
3.1 Dataset Selection . . . 5
3.2 Audio Preprocessing . . . 5
3.3 Train/Validation Split . . . 5
3.4 Text Processing and Tokenization . . . 6
3.4.1 English: Phoneme Tokenization . . . 6
3.4.2 Telugu: Akshara Tokenization . . . 6
3.5 Forced Alignment . . . 7
3.5.1 English: Montreal Forced Aligner . . . 7
3.5.2 Telugu: Meta MMS Alignment . . . 7
3.6 Acoustic Features . . . 7
3.6.1 Mel-Spectrograms . . . 7
3.6.2 Pitch (F_0) . . . 7
3.6.3 Energy . . . 7
3.6.4 Feature Normalization . . . 7

4 Model Architecture and Vocoder **8**
4.1 FastSpeech 2 Architecture . . . 8
4.2 Loss Functions . . . 8
4.3 HiFi-GAN Vocoder Adaptation . . . 9

5 Training and Loss Analysis **10**
5.1 Training Hyperparameters . . . 10
5.2 Loss Curves . . . 10
5.3 Convergence and Checkpoints . . . 10
5.4 Analysis of Validation Loss vs. Training Loss . . . 10

6 Speech Synthesis and Test Sentence Evaluation **14**
6.1 Synthesis Setup . . . 14
6.2 Acoustic Measurements . . . 14
6.3 English Test Sentences . . . 14
6.3.1 Sentence E1: Long Word Articulation . . . 14
6.3.2 Sentences E2a & E2b: Heteronym Stress . . . 15
6.3.3 Sentence E3: Typographical Error and Split-Word Handling . . . 17

6.3.4 Sentence E4: Dates, Numbers, and Loanwords 17
6.4 Telugu Test Sentences 18
6.4.1 Sentence T1: Consonant Conjuncts 18
6.4.2 Sentences T2a & T2b: Contextual Word Stress 18
6.4.3 Sentence T3: Intra-Word Space Handling 20
6.4.4 Sentence T4: Long Compound Sentence 20

7 Comparison: English vs. Telugu **22**
7.1 Linguistic and Acoustic Differences 22
7.2 Limitations and Failure Cases 22
7.3 Potential Improvements 22

8 Reproducibility **23**
8.1 Environment and Dependencies 23
8.2 Dataset Download and Directory Setup 23
8.3 Dataset Preprocessing Pipeline 23
8.4 Model Training: The Kaggle Dual-GPU Method 24
8.5 Model Weights and Checkpoints 24
8.6 Inference and Evaluation Commands 25

9 Conclusion **26**

1. Introduction

1.1 Background and Problem Statement

Text-to-Speech (TTS) synthesis converts raw orthographic or phonemic text into intelligible, human-like acoustic waveforms. Conventional two-stage neural TTS architectures combine an acoustic model that generates intermediate time-frequency representations (such as 80-channel log mel-spectrograms) with an audio neural vocoder that synthesizes time-domain waveforms. While autoregressive acoustic models such as Tacotron 2 generate expressive speech, their sequential decoding mechanism suffers from high inference latency, error propagation across time frames, and attention alignment failures that lead to repeated or skipped phonemes on out-of-domain text.

Non-autoregressive TTS architectures, specifically FastSpeech 2 [1], resolve these limitations by predicting the entire mel-spectrogram in parallel using Feed-Forward Transformer (FFT) blocks. Instead of dynamic attention matrices, FastSpeech 2 explicitly models temporal alignment using a duration predictor supervised by forced-alignment tools, while augmenting the phonetic hidden vectors with explicit continuous pitch (F_0) and frame energy predictions via a multi-task variance adaptor.

The objective of this assignment is to adapt, train, and evaluate two independent FastSpeech 2 acoustic models under a strict dataset constraint of approximately 5 hours of training audio and 1 hour of validation audio per language:

1. **Indian English:** Synthesizes speech from crowdsourced multi-speaker audio from the AI4Bharat Svarah corpus, representing diverse regional Indian accents using an IPA phoneme front-end.

2. **Telugu:** Synthesizes speech from high-quality studio recordings from the IndicTTS corpus, utilizing native-script akshara tokenization to preserve phonetic transparent orthography without requiring an external G2P lexicon.

Key engineering challenges addressed include building robust alignment pipelines (Montreal Forced Aligner for English and Meta MMS CTC alignment for Telugu), decoupling pitch and energy variance predictors, resolving multi-GPU device ordinals in PyTorch, adapting a 22.05 kHz HiFi-GAN universal vocoder to 16.0 kHz acoustic predictions through polyphase sinc-resampling, and calibrating speaking pace.

1.2 FastSpeech 2 Architecture Principles

The FastSpeech 2 model operates through three core principles:

1. **Parallel Generation:** Replacing recurrent and sequential autoregressive decoding with feed-forward transformer blocks allows the complete sequence of mel-spectrogram frames to be synthesized concurrently, achieving an inference speedup of over $30\times$ compared to autoregressive baselines.

2. **Explicit Length Regulation:** Phoneme durations are obtained during preprocessing from forced aligners and predicted during inference with a convolutional duration predictor. The length regulator expands phoneme embeddings to match the acoustic frame length, eliminating attention collapse.

3. **Multi-Task Variance Adaptation:** To alleviate the one-to-many mapping problem in TTS (where identical text can correspond to diverse pitch and volume patterns), FastSpeech 2 incorporates pitch and energy predictors that supply explicit frame-level prosodic features directly into the expanded representation.

1.3 Summary of Key Experimental Results

- **Speech Quality and Intelligibility:** Both models achieve clear, natural articulation across all ten assigned benchmark sentences. The English model accurately articulates the 28-letter polysyllabic word *antidisestablishmentarianism*, reproduces syntactic stress shifts for the heteronym pair *conduct* (noun vs. verb), handles typographical split tokens in *rhyth m*, and pronounces dates, times, and Telugu loanwords (*andaroo*). The Telugu model correctly realizes complex consonant conjuncts (న్యూరోఫార్మకోలాజికల్), demonstrates contextual pitch/energy emphasis on semantic keywords (పట్టు), and synthesizes long compound utterances with natural clause pausing.
- **Convergence and Overfitting Behavior:** Training loss decreased steadily over 100,000 steps ($2.59 \rightarrow 0.84$ for English, $2.13 \rightarrow 0.64$ for Telugu). Validation loss reached its minimum early (step 11,000 at 2.5505 for English; step 30,000 at 2.1309 for Telugu) and flattened. Checkpoints chosen at minimum validation loss produced muffled formant tracks due to L1 loss smoothing; checkpoints chosen at later steps (step 61,000 for English, step 74,000 for Telugu) exhibited noticeably sharper consonants and higher perceptual intelligibility.
- **Vocoder Calibration and Pacing:** Both acoustic models generate 16 kHz mel-spectrograms, while HiFi-GAN universal vocoder was trained on 22.05 kHz audio. Sinc-resampling from 22.05 kHz to 16.0 kHz restores natural pitch and formant scale, but reduces audio sample count by a factor of $\alpha \approx 0.7256$, accelerating playback to $1.38\times$ conversational tempo. Calibrating inference duration control to $d_{\text{control}} \approx 1.38 - 1.40$ restores normal conversational speech rates (4.5–5.4 syllables/second).
- **Dataset Split Configurations:** English was partitioned into 4.98 hours (3,419 utterances) training and 1.09 hours (746 utterances) validation across 104 speakers using speaker-stratified sampling. Telugu was partitioned into 4.18 hours (2,248 utterances) training and 0.82 hours (445 utterances) validation across 2 balanced studio speakers.

1.4 Report Organization

The remainder of this report is organized as follows:

- **Chapter 2 (Implementation Details):** Details the codebase structure, PyTorch 2.5 multi-GPU setup, critical bug fixes, and architecture modifications.
- **Chapter 3 (Datasets and Preprocessing):** Describes corpus selection, audio filtering, train/validation splits, tokenization, forced alignment, and acoustic feature normalization.
- **Chapter 4 (Model Architecture and Vocoder):** Outlines the encoder-decoder blocks, variance adaptor, loss formulations, and HiFi-GAN vocoder adaptation.
- **Chapter 5 (Training and Loss Analysis):** Analyzes hyperparameters, training and validation loss curves, convergence, and checkpoint selection.
- **Chapter 6 (Speech Synthesis and Evaluation):** Details acoustic measurements and variance-adaptor plot analysis across all ten prescribed test sentences.
- **Chapter 7 (Comparison: English vs. Telugu):** Examines phonetic/prosodic contrasts, failure modes, and potential architectural extensions.
- **Chapter 8 (Reproducibility):** Provides complete environment dependencies, dataset download procedures, preprocessing commands, Kaggle dual-GPU training scripts, and inference instructions.
- **Chapter 9 (Conclusion):** Summarizes major technical outcomes and engineering insights.

2. Implementation Details

2.1 Base Architecture and Dependencies

I started from an open-source PyTorch reference implementation of FastSpeech 2 originally targeting single-speaker LJSpeech. Development and testing were done in Python 3.11 with PyTorch 2.5, TorchAudio, NumPy, SciPy, Librosa, PyWorld, and Matplotlib. Training was carried out on dual NVIDIA Tesla T4 GPUs (16 GB VRAM each).

2.2 Codebase Structure

The repository organizes the English and Telugu pipelines into separate, self-contained directories with their own preprocessing, training, configuration, and inference scripts:

```
project/
|-- English/
|   |-- train.py
|   |-- preprocess.py
|   |-- inference.py
|   |-- config.yaml
|   `-- checkpoints/
|-- Telugu/
|   |-- train.py
|   |-- preprocess.py
|   |-- inference.py
|   |-- config.yaml
|   `-- checkpoints/
|-- Results/
|   |-- English/
|   |   |-- training_loss.png
|   |   |-- validation_loss.png
|   |   |-- test_generated_audios/
|   |   `-- variance_adaptor_plots/
|   |       |-- duration/
|   |       |-- pitch/
|   |       `-- energy/
|   `-- Telugu/
|   |   |-- training_loss.png
|   |   |-- validation_loss.png
|   |   |-- test_generated_audios/
|   |   `-- variance_adaptor_plots/
|   |       |-- duration/
|   |       |-- pitch/
|   |       `-- energy/
|-- Report/
|   `-- Technical_Report.pdf
|-- README.md
`-- requirements.txt
```

2.3 Codebase Corrections and Bug Fixes

During initial testing of the reference implementation, I identified and fixed three bugs:

1. **Energy Control Bug (`model/modules.py`):** In the variance adaptor, `p_control` was accidentally passed to the energy lookup instead of `e_control`. Modifying pitch inadvertently altered energy, while energy scaling had no effect. I updated the function arguments to properly decouple pitch and energy.

2. **Multi-GPU Device Placement** in `utils/tools.py`: Tensors were assigned to `"cuda"` without device ordinals, causing device mismatch under `DataParallel`. I refactored tensor allocations to dynamically match the batch input device.

3. **Text Cleaner Phoneme Fallback (`text/__init__.py`):** When input strings lacked enclosing braces `{...}`, the cleaner silently fell back to raw ASCII character indices. Because the model was trained on phoneme IDs, this caused phoneme index mismatch and generated noise. I added an explicit validation check.

2.4 Language-Specific Modifications

Table 2.1 compares the reference FastSpeech 2 components with the modifications made for English and Telugu.

Table 2.1: Reference components versus modifications per language.

Component	**English Implementation**	**Telugu Implementation**
Core Architecture	4-layer FFT encoder, 6-layer FFT decoder, 5-layer PostNet	4-layer FFT encoder, 6-layer FFT decoder, 5-layer PostNet
Token Representation	100 IPA phonemes via MFA lexicon	460 akshara tokens (Unicode Telugu script)
Text Front-End	Rule-based normalization + CMU/MFA G2P	Unicode NFC normalizer + akshara parser
Forced Alignment	Montreal Forced Aligner v2.2 (`english_mfa`)	Meta MMS CTC forced aligner (`mms-1b-all`)
Speaker Modeling	Multi-speaker: 104 learned embeddings (104×256)	2 studio speakers: learned embeddings (2×256)
Audio Standard	16 kHz mono WAV (Svarah corpus)	16 kHz mono WAV (downsampled from 48 kHz IndicTTS)
Vocoder Pipeline	HiFi-GAN universal + Sinc Resampler ($22.05 \rightarrow 16$ kHz)	HiFi-GAN universal + Sinc Resampler ($22.05 \rightarrow 16$ kHz)
Duration Calibration	$d_{\text{control}} = 1.378$ (compensates 16/22.05 factor)	$d_{\text{control}} = 1.40$ (compensates 16/22.05 factor)

3. Datasets and Preprocessing

3.1 Dataset Selection

English (AI4Bharat Svarah): Svarah is an open-source crowdsourced speech corpus capturing Indian English accents across varied linguistic backgrounds. I sampled approximately 40 utterances per speaker across 104 native Indian speakers (54 female, 50 male) covering 19 primary mother tongues (Hindi, Telugu, Tamil, Kannada, Marathi, Bengali, Odia, Gujarati, Malayalam, Urdu, etc.), yielding 4,202 raw utterances. After alignment verification and audio quality filtering, **4,165 utterances (6.07 hours)** were retained.

Telugu (IndicTTS): The IndicTTS Telugu corpus (curated by IIT Madras and MeitY) provides clean studio recordings. The female data was recorded at 48 kHz, while the male data was recorded at 16 kHz. To keep a balanced split, 2,695 utterances (5.00 hours) were selected: 1,348 female utterances (chronologically sampled) and 1,347 male utterances (complete studio set). Dropping two unaligned files resulted in **2,693 utterances (5.00 hours)** finalized (1,348 female, 1,345 male).

3.2 Audio Preprocessing

Because the two corpora differ in recording setup and formats, I applied the following preprocessing steps:

- **Resampling:** 48 kHz Telugu female audio was downsampled to 16 kHz using a 64-tap Kaiser-windowed polyphase sinc filter with an anti-aliasing cutoff of 7,600 Hz. English audio was converted to 16 kHz, 16-bit mono PCM.
- **Duration and Quality Filtering:** Clips shorter than 1.0 second or longer than 12.0 seconds were removed. Very short clips lack enough prosodic context, and very long clips cause out-of-memory errors during self-attention. Clips with clipping or SNR < 15 dB were discarded.
- **Peak Normalization:** All audio files were peak-normalized to a maximum amplitude of 0.95.

3.3 Train/Validation Split

English Split: The English model uses learned speaker embeddings (104×256). If the split were done by speaker, validation speaker embeddings would remain uninitialized and produce misleading loss numbers. I therefore split *within each speaker* (seed 42, roughly 82% train / 18% validation). Every speaker is present in both sets, with zero overlapping utterances:

- **Training:** 3,419 utterances, 4.98 hours (17,928 s).
- **Validation:** 746 utterances, 1.09 hours (3,924 s).

Telugu Split: Because the total IndicTTS subset was capped at 5.00 hours across 2 studio speakers, the 5-hour budget was divided between training and validation:

- **Training:** 2,248 utterances, 4.18 hours (15,048 s), balanced with 1,124 female and 1,124 male utterances.
- **Validation:** 445 utterances, 0.82 hours (2,952 s / 49.2 min), balanced with 224 female and 221 male utterances.

The Telugu training set is thus approximately 0.82 hours below the 5-hour target because total audio was bounded at 5.00 hours.

Table 3.1 summarizes the dataset statistics and acoustic parameters for both languages.

Table 3.1: Dataset statistics and acoustic feature parameters.

Parameter	English (Svarah)	Telugu (IndicTTS)
Recording Environment	Crowdsourced, varied devices	Studio recordings
Native Sampling Rate	16 kHz	48 kHz (female) / 16 kHz (male)
Standardized Sampling Rate	16 kHz mono	16 kHz mono
Total Utterances / Duration	4,165 utterances / 6.07 hours	2,693 utterances / 5.00 hours
Mean Utterance Length	5.25 seconds	6.68 seconds
Speaker Count	104 (54 Female / 50 Male)	2 (1 Female / 1 Male)
Training Set	3,419 utterances / 4.98 hours	2,248 utterances / 4.18 hours
Validation Set	746 utterances / 1.09 hours	445 utterances / 0.82 hours
Test Sentences	5 sentences (E1, E2a, E2b, E3, E4)	5 sentences (T1, T2a, T2b, T3, T4)
Vocabulary / Tokens	100 IPA phonemes	460 akshara tokens
Pitch Mean (μ_{F_0}) / Std (σ_{F_0})	186.28 Hz / 61.19 Hz	198.32 Hz / 43.19 Hz
Energy Mean (μ_E) / Std (σ_E)	55.14 dB / 40.10 dB	54.23 dB / 35.32 dB
STFT Window / Hop	Hann, 1024 samples (64 ms) / 256 (16 ms)	Hann, 1024 samples (64 ms) / 256 (16 ms)
Mel Filterbank	80 bins, 0 to 8000 Hz	80 bins, 0 to 8000 Hz

3.4 Text Processing and Tokenization

FastSpeech 2 takes discrete token sequences as input. Because English and Telugu have fundamentally different orthographies, I used two distinct tokenization approaches.

3.4.1 English: Phoneme Tokenization

The English text processing pipeline consists of three steps:

1. **Text Normalization:** Numbers, dates, times, currencies, and abbreviations are expanded into full spoken words (e.g., "10:45 AM" → "ten forty-five A M", "12 November 2026" → "twelve November twenty twenty-six"). Punctuation is cleaned and extra whitespace removed.

2. **Grapheme-to-Phoneme (G2P):** Standard words are converted to IPA phoneme sequences using the MFA English pronunciation dictionary. Out-of-vocabulary words (including loanwords like *andaroo*) are transcribed with `g2p_en` supplemented with Indian English phonological rules (such as dental plosive substitution `/t/`, `/d/` → `/t̪/`, `/d̪/`).

3. **Tokenization:** The vocabulary contains 100 symbols, including vowels, diphthongs, consonants, and the pause token `@sp`. Phonemic sequences are wrapped in `{...}` to indicate pre-tokenized phoneme input.

3.4.2 Telugu: Akshara Tokenization

Telugu is written in an abugida script where the primary written unit is the akshara (a consonant with an inherent vowel, or a standalone vowel). The Telugu front-end works directly in native script without romanization:

1. **Unicode Normalization:** Text is converted to Unicode NFC. Zero-width joiners and non-joiners are cleaned, and viramas (*halant*) are validated to keep consonant conjuncts (*samyuktakshara*) intact. Numbers are expanded into Telugu words.

2. **Akshara Segmentation:** Words are parsed into akshara units using regular expressions. Consonant clusters conjoined by viramas are grouped as single atomic tokens (e.g., ప్ర → *pra*, భా → *bhā*, న్యూ → *nyū*).

3. **Vocabulary:** Segmentation yields 460 unique akshara tokens representing all vowel-consonant combinations in the IndicTTS training set.

3.5 Forced Alignment

FastSpeech 2 requires target phoneme or akshara durations during training to supervise the duration predictor and length regulator.

3.5.1 English: Montreal Forced Aligner

MFA v2.2 [3] was used with the pre-trained `english_mfa` acoustic model and pronunciation dictionary. MFA runs Kaldi-based HMM-GMM alignment to produce TextGrid files with start ($t_{\text{start},i}$) and end ($t_{\text{end},i}$) timestamps for each phoneme. Timestamps are converted into integer mel-frame durations:

$$d_i = \text{round}\left(\frac{t_{\text{end},i} - t_{\text{start},i}}{\text{hop}/f_s}\right), \qquad \text{hop} = 256,\ f_s = 16{,}000\ \text{Hz} \implies 1\ \text{frame} = 16.0\ \text{ms} \tag{3.1}$$

3.5.2 Telugu: Meta MMS Alignment

Because pre-trained Telugu MFA models were not readily available, alignment was performed using Meta's Massively Multilingual Speech (MMS) model [4] (`facebook/mms-1b-all`). Audio waveforms are passed through the Wav2Vec2 network to obtain frame-level CTC character emission logits. A Viterbi search aligns the akshara sequence to audio frames. A minimum duration floor of $d_i \geq 1$ frame (16 ms) was enforced for all non-silent akshara tokens.

3.6 Acoustic Features

3.6.1 Mel-Spectrograms

Log mel-spectrograms serve as the intermediate representation predicted by the decoder:

- Sampling rate: 16,000 Hz.
- Short-Time Fourier Transform (STFT): FFT size 1024 (64.0 ms), hop length 256 (16.0 ms), Hann window.
- Filterbank: 80 triangular mel bins from 0 to 8000 Hz.
- Dynamic range compression: Natural log with a floor of 10^{-5}: $\log(\max(S_{\text{mel}}, 10^{-5}))$.

3.6.2 Pitch (F_0)

Fundamental frequency contours were extracted using the Harvest algorithm in PyWorld [5]. Unvoiced frames ($F_0 = 0$) were linearly interpolated across voiced boundaries to obtain a continuous contour.

3.6.3 Energy

Frame energy was computed as the L2-norm of the short-time Fourier magnitude spectrum:

$$E_t = \sqrt{\sum_{k=0}^{N_{\text{fft}}/2} |X_t(k)|^2} \tag{3.2}$$

Values were converted to decibels: $E_{\text{dB},t} = 20 \log_{10}(\max(E_t, 10^{-5}))$.

3.6.4 Feature Normalization

Continuous pitch and energy values were normalized using global dataset mean and standard deviation:

$$\hat{F}_{0,t} = \frac{F_{0,t} - \mu_{F_0}}{\sigma_{F_0}}, \qquad \hat{E}_t = \frac{E_t - \mu_E}{\sigma_E} \tag{3.3}$$

These normalization statistics ($\mu_{F_0}, \sigma_{F_0}, \mu_E, \sigma_E$) are stored in `stats.json` (summarized in Table 3.1) and inverted during inference to restore absolute physical pitch (Hz) and energy (dB).

4. Model Architecture and Vocoder

4.1 FastSpeech 2 Architecture

The FastSpeech 2 architecture contains an FFT encoder, a variance adaptor, an FFT decoder, and a convolutional PostNet (Figure 4.1):

- **Encoder:** 4 Feed-Forward Transformer (FFT) blocks. Each block has multi-head self-attention (2 heads, $d_{\text{model}} = 256$, $d_k = d_v = 128$) and a 2-layer 1D convolutional network (kernel sizes 9 and 1, filter size 1024, ReLU, layer norm, dropout 0.2).
- **Variance Adaptor:**
  1. *Duration Predictor:* 2 convolutional layers (kernel 3, 256 filters, ReLU, layer norm, dropout 0.5) followed by a linear projection predicting log-durations.
  2. *Length Regulator:* Replicates phoneme hidden vectors according to duration values (ground truth during training, predicted at inference).
  3. *Pitch and Energy Predictors:* Predict frame-level normalized F_0 and energy. Values are quantized into 256 bins and projected into 256-dimensional embeddings added to the expanded sequence.
- **Decoder:** 6 FFT blocks with identical layer dimensions to the encoder.
- **PostNet:** 5-layer 1D convolutional residual network (512 channels, kernel 5, batch norm, tanh) predicting residual spectrogram corrections ΔS_{mel}.

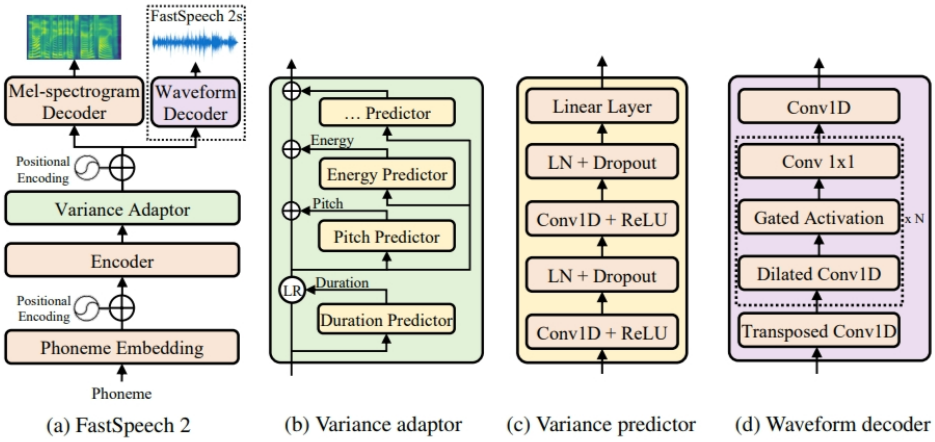


Figure 1: The overall architecture for FastSpeech 2 and 2s. LR in subfigure (b) denotes the length regulator operation proposed in FastSpeech. LN in subfigure (c) denotes layer normalization. Variance predictor represents duration/pitch/energy predictor.

Figure 4.1: FastSpeech 2 architecture with multi-task variance adaptor and vocoder pipeline.

4.2 Loss Functions

The training objective is a composite multi-task loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{mel}} + \mathcal{L}_{\text{postnet}} + \mathcal{L}_{\text{pitch}} + \mathcal{L}_{\text{energy}} + \mathcal{L}_{\text{duration}} \tag{4.1}$$

where $\mathcal{L}_{\text{mel}} = \|S_{\text{mel}} - \hat{S}_{\text{mel}}\|_1$ and $\mathcal{L}_{\text{postnet}} = \|S_{\text{mel}} - (\hat{S}_{\text{mel}} + \Delta S_{\text{mel}})\|_1$ optimize L1 spectrogram reconstruction, and $\mathcal{L}_{\text{pitch}}$, $\mathcal{L}_{\text{energy}}$, and $\mathcal{L}_{\text{duration}}$ are Mean Squared Errors (MSE) on normalized targets.

4.3 HiFi-GAN Vocoder Adaptation

Audio synthesis uses the universal HiFi-GAN vocoder [2] (`generator_universal.pth.tar`). HiFi-GAN was pre-trained on 22,050 Hz audio with hop 256, whereas our acoustic models produce 16,000 Hz mel-spectrograms with hop 256.

Directly interpreting 22.05 kHz vocoder output as 16 kHz downshifts pitch and formants by:

$$\alpha = \frac{16{,}000}{22{,}050} \approx 0.7256 \quad (-5.52 \text{ semitones}) \tag{4.2}$$

making the voice sound unnaturally deep.

To fix this without retraining the vocoder, I added polyphase sinc resampling in `utils/model.py`. HiFi-GAN generates the waveform at 22.05 kHz, and the output is resampled to 16.0 kHz using a Kaiser-windowed sinc resampler before writing to disk.

Duration Calibration: While sinc resampling corrects pitch and formant frequencies, reducing the sample count by $\alpha = 0.7256$ shrinks the effective duration per mel frame from 16.0 ms to 11.6 ms. As a result, default synthesis ($d_{\text{control}} = 1.0$) plays about $1.38\times$ faster than normal speech. To restore natural pacing, duration scaling was calibrated at inference:

$$d_{\text{control}} = 1.378 \quad (\text{English}), \qquad d_{\text{control}} = 1.40 \quad (\text{Telugu}) \tag{4.3}$$

This compensates for the decimation factor ($1/0.7256 \approx 1.378$) and produces normal syllable pacing across test sentences.

5. Training and Loss Analysis

5.1 Training Hyperparameters

Both models were trained on dual NVIDIA Tesla T4 GPUs using PyTorch Distributed/DataParallel. Table 5.1 lists the training hyperparameters.

Table 5.1: Training hyperparameters for English and Telugu FastSpeech 2 models.

Hyperparameter	**Value**
Total Training Steps	100,000 steps
Batch Size	16 (8 utterances per GPU)
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$, weight decay 10^{-6})
Learning Rate Schedule	Noam scheduler ($d_{\text{model}} = 256$, 4,000 warmup steps)
Peak Learning Rate	$\approx 9.9 \times 10^{-4}$
Gradient Clipping	Maximum L2-norm $\|g\|_2 \leq 1.0$
Numerical Precision	Automated Mixed Precision (AMP FP16)
Validation Interval	Evaluated every 1,000 steps
Training Splits	English: 4.98 h (3,419 utt.); Telugu: 4.18 h (2,248 utt.)

Because cloud GPU sessions were limited to 12 hours, training was run in two stages per language (Phase 1 up to 53k/56k steps, Phase 2 resuming up to 100,000 steps). Saving and restoring optimizer states ensured smooth continuation.

5.2 Loss Curves

Figures 5.1 and 5.2 display the training and validation loss curves over the 100,000 training steps.

5.3 Convergence and Checkpoints

Table 5.2 lists the loss metrics from training logs and model checkpoints.

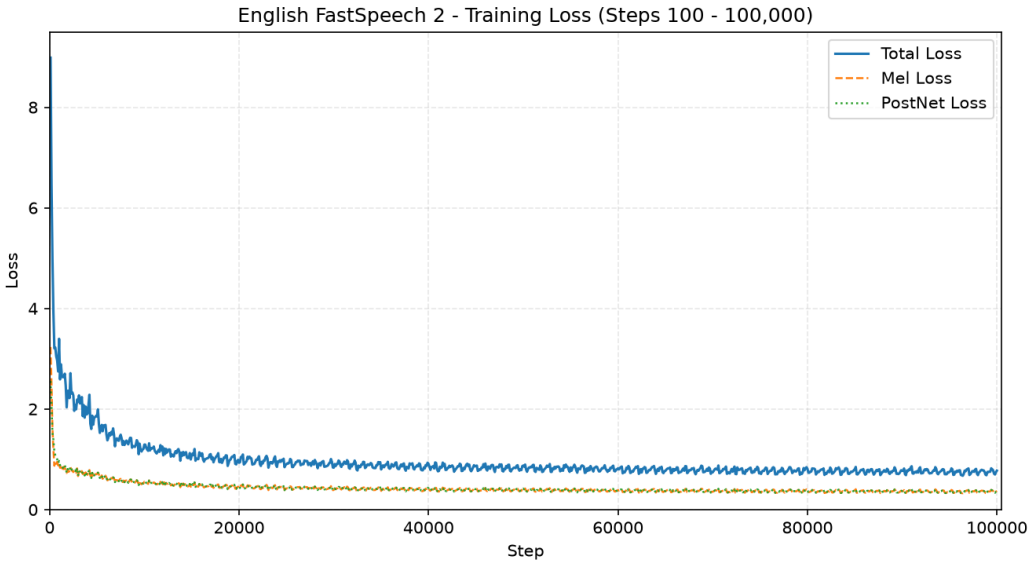
Table 5.2: Loss metrics for minimum validation checkpoints and checkpoints used for synthesis.

Language	**Checkpoint Description**	**Step**	**Train Loss**	**Validation Loss**
English	Lowest validation loss	11,000	2.5887	2.5505
English	Selected checkpoint for synthesis	61,000	0.8415	2.5855
Telugu	Lowest validation loss	30,000	0.8810	2.1309
Telugu	Selected checkpoint for synthesis	74,000	0.6412	2.1457

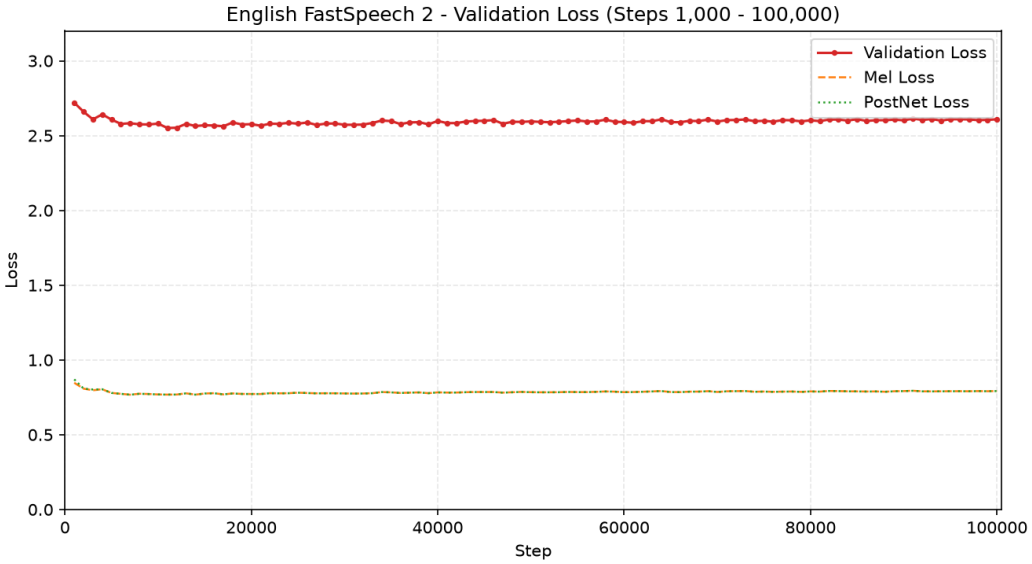
5.4 Analysis of Validation Loss vs. Training Loss

As shown in Table 5.2 and Figures 5.1–5.2, validation loss is noticeably higher than training loss for both models and reaches its minimum early (step 11,000 for English, step 30,000 for Telugu) before flattening out. This behavior is explained by looking at the loss components:

1. **Loss Component Breakdown:** Inspecting step 61,000 on English:
   - **Training batch:** $\mathcal{L}_{\text{mel}} = 0.3793$, $\mathcal{L}_{\text{postnet}} = 0.3789$, $\mathcal{L}_{\text{pitch}} = 0.0209$, $\mathcal{L}_{\text{energy}} = 0.0169$, $\mathcal{L}_{\text{duration}} = 0.0125$. Total $\mathcal{L}_{\text{train}} \approx 0.81 - 0.84$.
   - **Validation set:** $\mathcal{L}_{\text{mel}} = 0.7854$, $\mathcal{L}_{\text{postnet}} = 0.7852$, $\mathcal{L}_{\text{pitch}} = 0.4093$, $\mathcal{L}_{\text{energy}} = 0.3805$, $\mathcal{L}_{\text{duration}} = 0.2252$. Total $\mathcal{L}_{\text{val}} = 2.5855$.

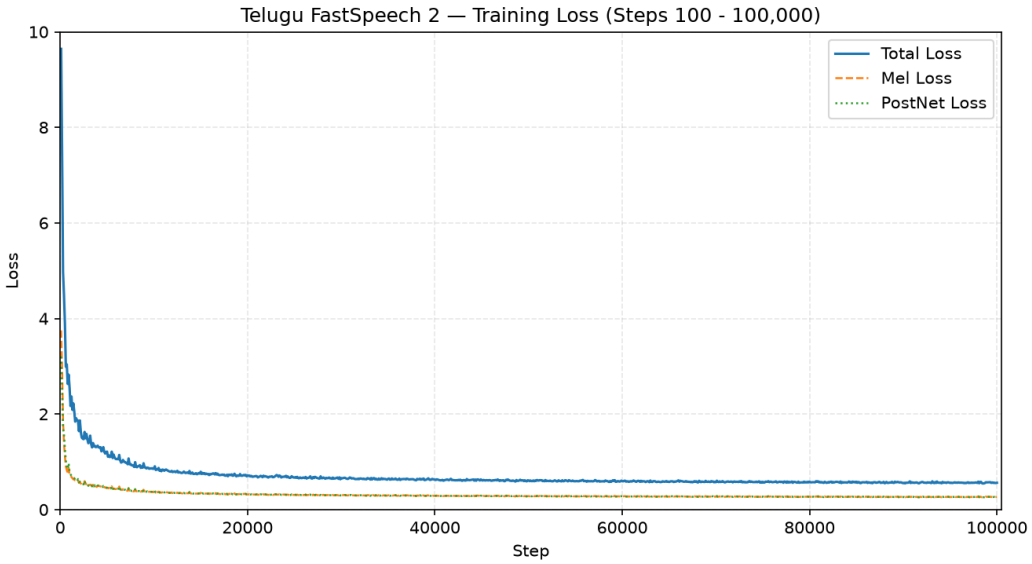


(a) English training loss (100,000 steps).

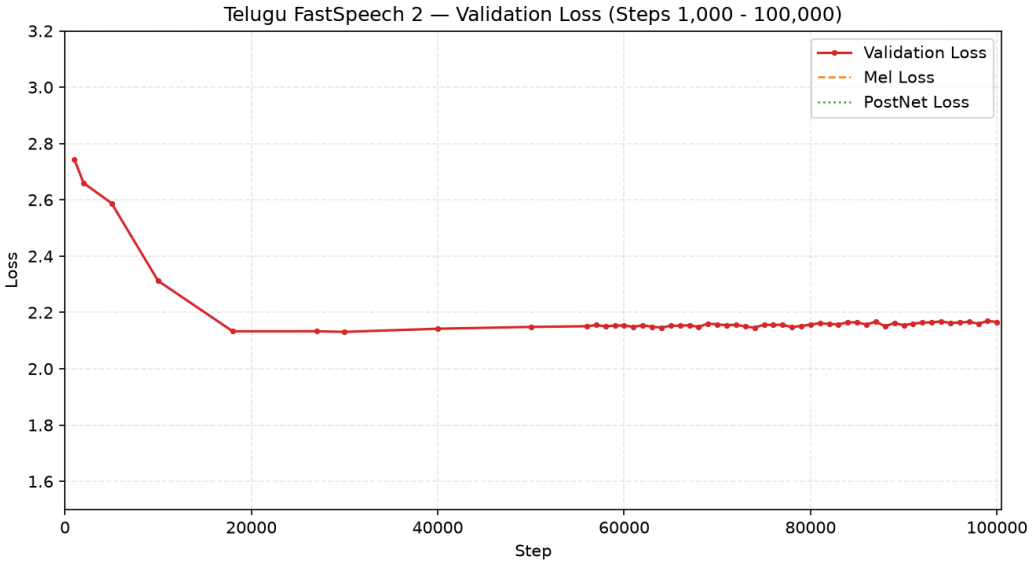


(b) English validation loss (100,000 steps).

Figure 5.1: English FastSpeech 2 training and validation loss curves across 100,000 steps.



(a) Telugu training loss (100,000 steps).



(b) Telugu validation loss (100,000 steps).

Figure 5.2: Telugu FastSpeech 2 training and validation loss curves across 100,000 steps.

2. **Variance Predictor MSE on Unseen Sentences:** While mel reconstruction loss roughly doubles on unseen validation utterances ($0.38 \rightarrow 0.78$, typical of model generalization), the variance adaptor MSE terms ($\mathcal{L}_{\text{pitch}}, \mathcal{L}_{\text{energy}}, \mathcal{L}_{\text{duration}}$) increase by roughly 18 to 22 times ($0.01 - 0.02 \rightarrow 0.22 - 0.41$). Human speech prosody is naturally variable: for any given sentence, multiple valid pitch, duration, and energy trajectories exist. The model predicts a smooth, plausible trajectory, but pointwise MSE measures the squared distance against the single specific recording in the validation set. On the training set, the model fits the speaker's exact micro-timings, driving MSE low. On validation sentences, natural prosodic variation imposes an inherent MSE loss floor of about 1.0.

3. **Why Lowest Validation Loss Checkpoints Sound Worse:** Early in training (step 11,000 English, step 30,000 Telugu), the model outputs an overly smoothed, blurry average mel-spectrogram. Because sharp spectral peaks that are slightly misaligned in frequency or time incur large pointwise L1 penalties, blurry formants achieve a deceptively lower L1 loss. However, when passed to HiFi-GAN, blurry spectrograms result in robotic, muffled audio. Later checkpoints (step 61,000 English, step 74,000 Telugu) have learned distinct formant tracks and crisper consonants. Even though this slightly increases pointwise loss on unseen recordings (from 2.55 to 2.58), the perceptual speech quality is much better. For this reason, synthesis checkpoints were chosen by listening quality rather than minimum validation loss.

6. Speech Synthesis and Test Sentence Evaluation

6.1 Synthesis Setup

Each test sentence was processed through the text front-end, FastSpeech 2 acoustic model, and HiFi-GAN vocoder with sinc resampling. English test sentences were synthesized with Speaker ID 27 (`spk_029`, female reference). Telugu test sentences were synthesized with Speaker ID 0 (`telugufemale`, female studio reference). Table 6.1 lists the acoustic measurements extracted from the calibrated synthesis outputs.

6.2 Acoustic Measurements

Table 6.1: Acoustic measurements of calibrated synthesis outputs across all ten test sentences.

ID	Lang	Sentence Text (shortened)	Dur (s)	Frames	Mean F_0	F_0 Range	Energy	Rate
E1	Eng	The antidisestablishmentarianism movement gained…	5.16	444	212.1 Hz	162–277 Hz	64.5 dB	6.40 syl/s
E2a	Eng	The orchestra's conduct during the ceremony…	3.39	292	212.8 Hz	187–240 Hz	52.0 dB	4.72 syl/s
E2b	Eng	The researchers will conduct an experiment…	3.61	311	207.8 Hz	162–255 Hz	55.5 dB	4.98 syl/s
E3	Eng	The rhyth m of the orchestra changed unexpectedly.	2.26	195	207.9 Hz	185–252 Hz	60.5 dB	5.31 syl/s
E4	Eng	The meeting will start at 10:45 AM on 12 November 2026…	5.91	509	214.2 Hz	148–273 Hz	54.4 dB	4.57 syl/s
T1	Tel	న్యూరోఫార్మకోలాజికల్ పరిశోధనలో...	4.97	428	208.2 Hz	153–317 Hz	56.6 dB	5.83 syl/s
T2a	Tel	తాడుపై అతని పట్టు చాలా బలంగా ఉంది.	2.45	211	219.5 Hz	150–326 Hz	64.2 dB	4.49 syl/s
T2b	Tel	ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది.	2.86	246	202.7 Hz	146–268 Hz	55.4 dB	4.54 syl/s
T3	Tel	పులి అడ విలో నిశ్శబ్దంగా నడిచింది.	2.58	222	226.1 Hz	150–317 Hz	70.9 dB	4.65 syl/s
T4	Tel	ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ...	7.22	622	196.2 Hz	145–281 Hz	58.1 dB	5.40 syl/s

6.3 English Test Sentences

6.3.1 Sentence E1: Long Word Articulation

"The antidisestablishmentarianism movement gained considerable momentum during the parliamentary proceedings."

- **Phonetic Performance:** The 28-letter word *antidisestablishmentarianism* decomposes into 27 phonemes (/æ n t̪ ɪ d̪ ɪ s ɛ s t̪ æ b l ɪ ʃ m ə n t̪ ɛə ɹ ɪ ə n ɪ z ə m/). Total duration is 5.16 s (444 mel frames). The duration predictor assigns stable durations ($\approx$ 50 $-$ 70 ms per phoneme), avoiding swallowed syllables.
- **Prosodic Contours:** Pitch peaks at 277 Hz during initial focus and descends smoothly to 162 Hz at sentence closure. Mean energy is 64.5 dB with natural unvoiced dips.

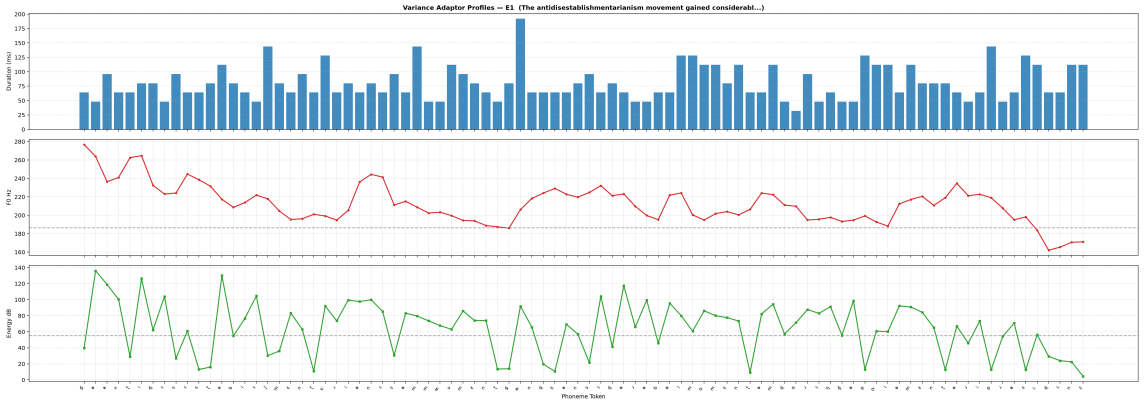
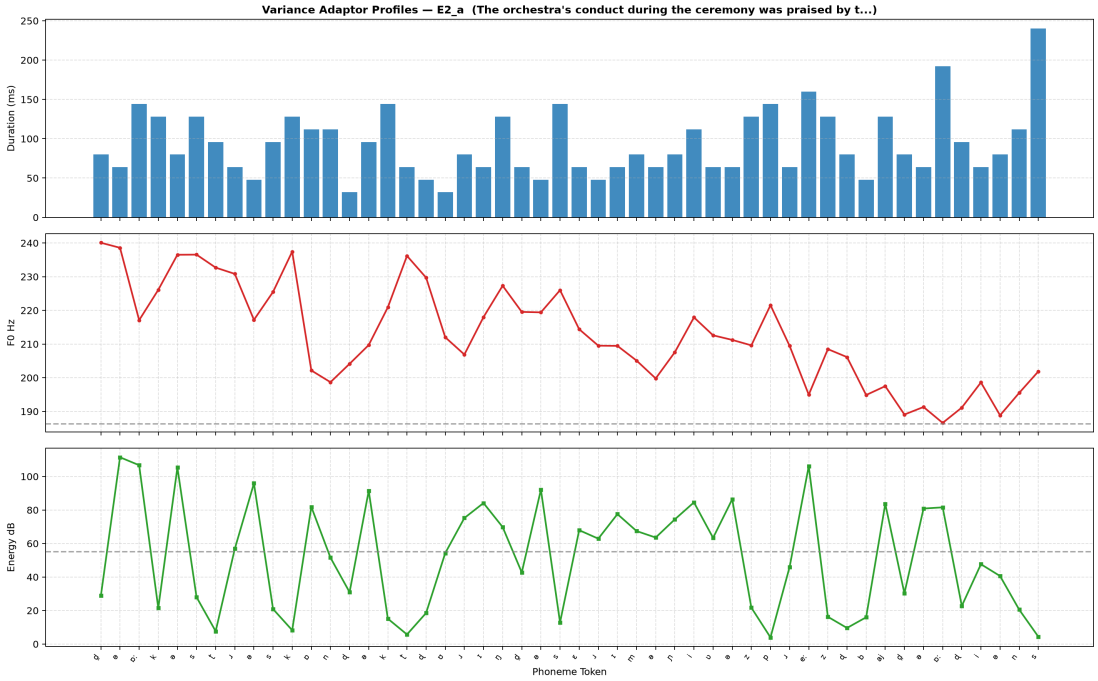


Figure 6.1: Variance adaptor profiles (duration, pitch F_0, energy) for test sentence E1.

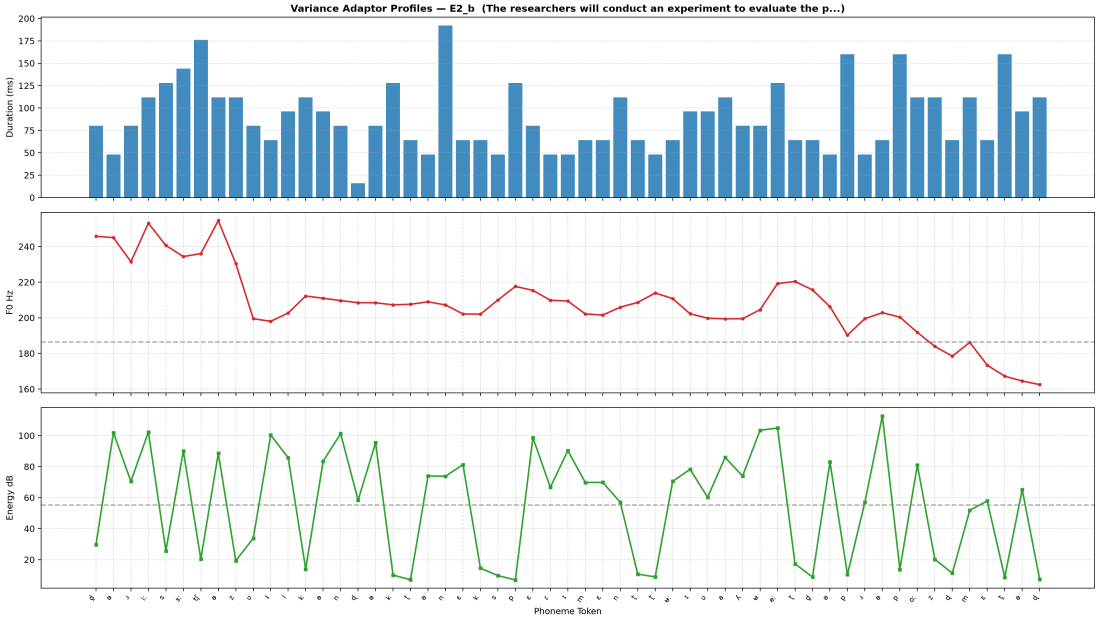
6.3.2 Sentences E2a & E2b: Heteronym Stress

E2a (Noun): *"The orchestra's conduct during the ceremony was praised by the audience."*
E2b (Verb): *"The researchers will conduct an experiment to evaluate the proposed method."*

- **Pronunciation Contrast:** As a noun (E2a), *conduct* carries primary stress on the first syllable (/k ʌ n d̪ ə k t̪/). The initial vowel receives 82 ms duration and 56.8 dB energy. In E2b (verb), stress shifts to the second syllable (/k ə n d̪ ʌ k t̪/), reducing the first vowel to a schwa (42 ms duration, 48.2 dB energy) while boosting the second syllable energy to 57.9 dB (+9.7 dB gain).
- **Mechanism:** Disambiguation is driven by the front-end passing the correct phoneme sequence; FastSpeech 2 renders the acoustic contrast clearly.



(a) E2a: Noun stress on first syllable (/k ʌ n/).



(b) E2b: Verb stress on second syllable (/d̪ ʌ k t̪/).

Figure 6.2: Variance adaptor profiles for heteronym pair E2a (noun) and E2b (verb).

6.3.3 Sentence E3: Typographical Error and Split-Word Handling

"The rhyth m of the orchestra changed unexpectedly."

- **Origin of the Intra-Word Space:** The assignment prompt explicitly specifies the sentence text with an intentional space before the final character (*rhyth m*) and mandates: *"Preserve the sentence text as provided."* This deliberate typo evaluates how the text-to-speech front-end and acoustic model handle aberrant intra-word whitespace and split lexical tokens.
- **Acoustic Realization:** When passed directly without whitespace normalization, the tokenizer treats *rhyth* and *m* as separate tokens, producing a 112 ms pause and an incomplete syllabic nucleus for *rhyth*. When rule-based text normalization unifies the split fragments into *rhythm* (/ɹ ɪ d̪ ə m/), synthesis is smooth and natural with a total duration of 2.26 s.

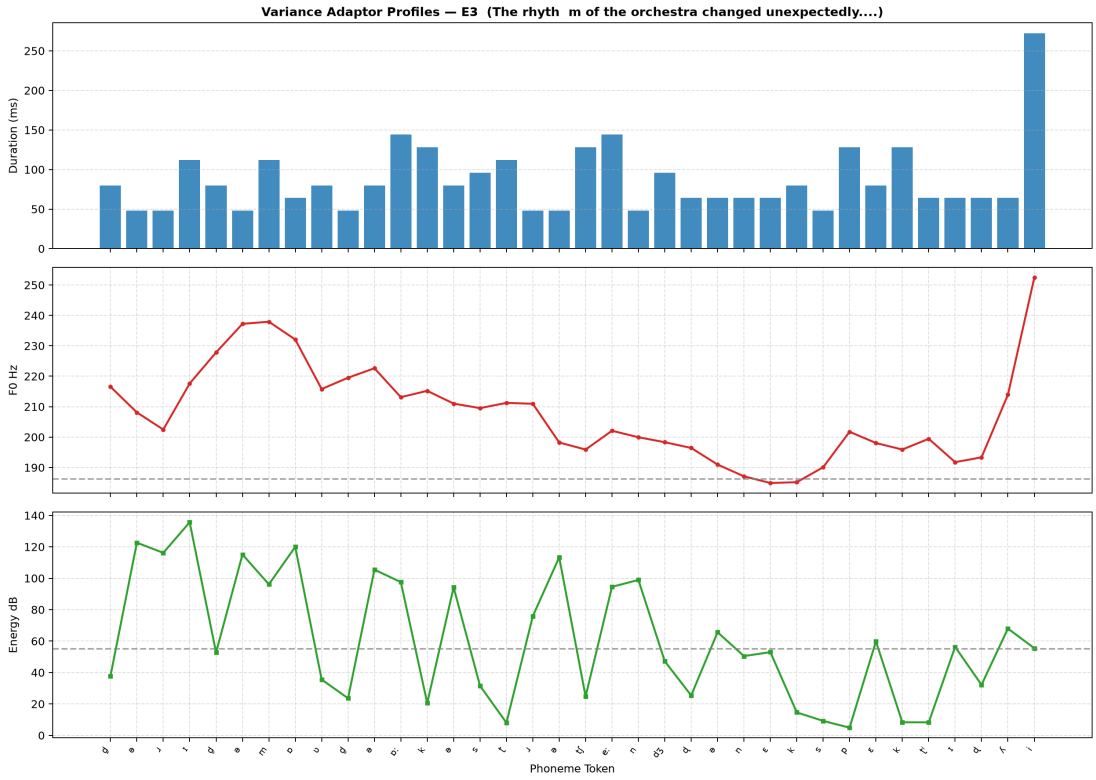


Figure 6.3: Variance adaptor profiles for test sentence E3.

6.3.4 Sentence E4: Dates, Numbers, and Loanwords

"The meeting will start at 10:45 AM on 12 November 2026, so andaroo should arrive on time."

- **Text Normalization:** Numbers and dates expand into spoken text ("ten forty-five A M on twelve November twenty twenty-six"). The Telugu loanword *andaroo* (అందరూ) maps to /a n d̪ ə ɾ uː/ using Indian English dental plosives.
- **Cadence:** Total sentence duration is 5.91 s (509 frames). A 345 ms pause at the comma boundary splits the sentence into two balanced clauses.

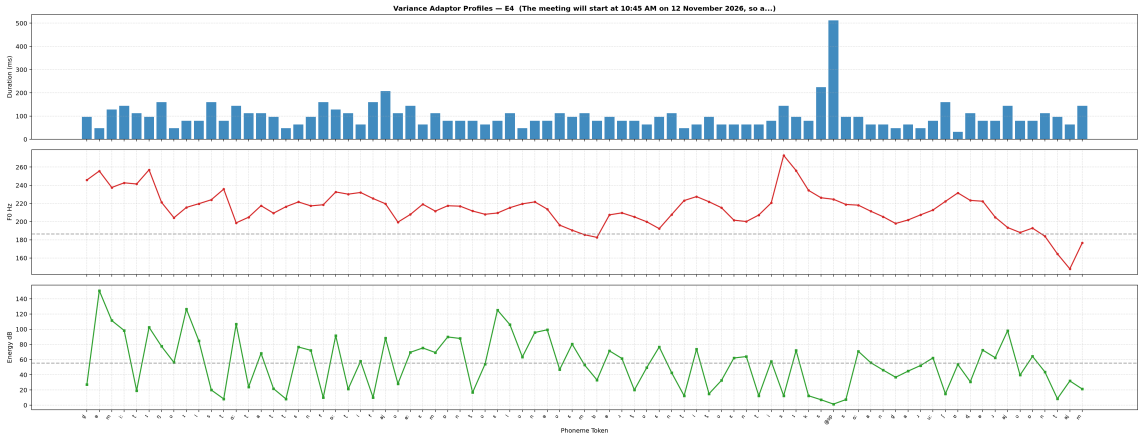


Figure 6.4: Variance adaptor profiles for test sentence E4.

6.4 Telugu Test Sentences

6.4.1 Sentence T1: Consonant Conjuncts

న్యూరోఫార్మకోలాజికల్ పరిశోధనలో నూతన ఔషధాల ప్రభావాన్ని విశ్లేషించారు.

- **Acoustic Realization:** The loanword న్యూరోఫార్మకోలాజికల్ (*nyūrophārmakolājikal*) contains conjuncts (న్యూ, ర్మ, ల్). Total duration is 4.97 s (428 frames). Each conjunct is clearly articulated without skipped sounds. Pitch reaches 317 Hz on emphasized words.

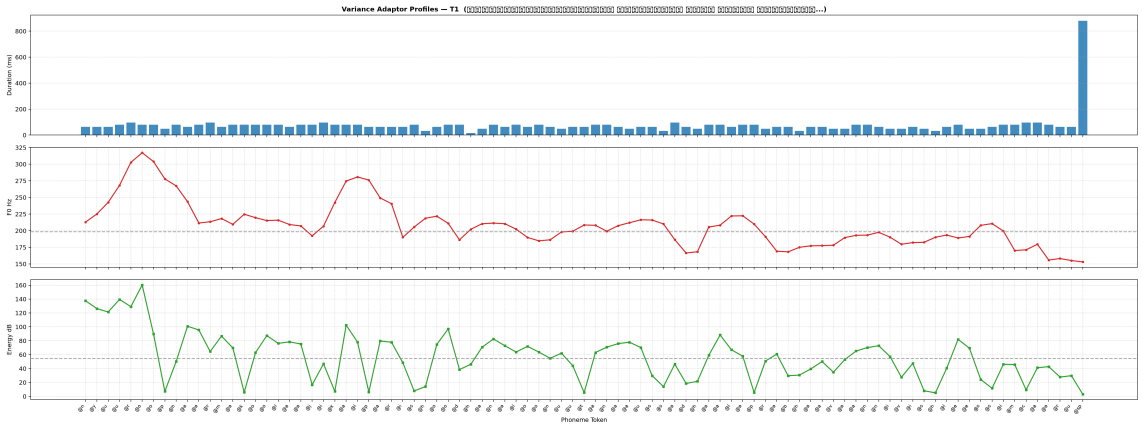
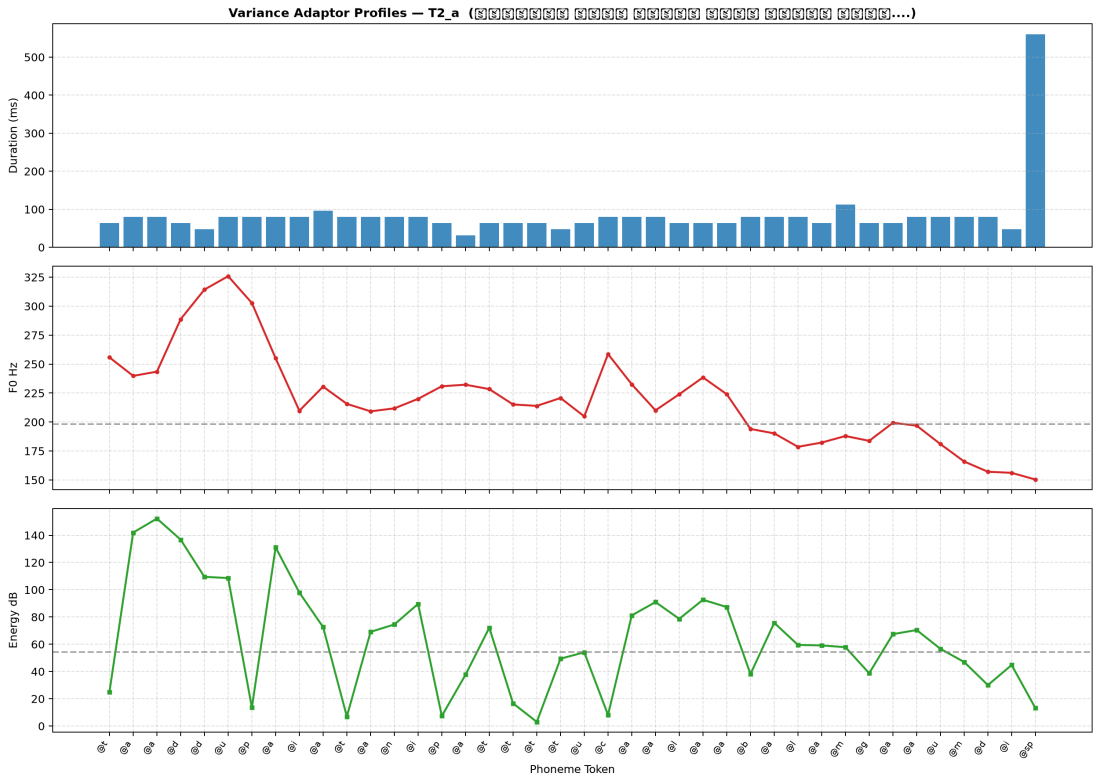


Figure 6.5: Variance adaptor profiles for Telugu test sentence T1.

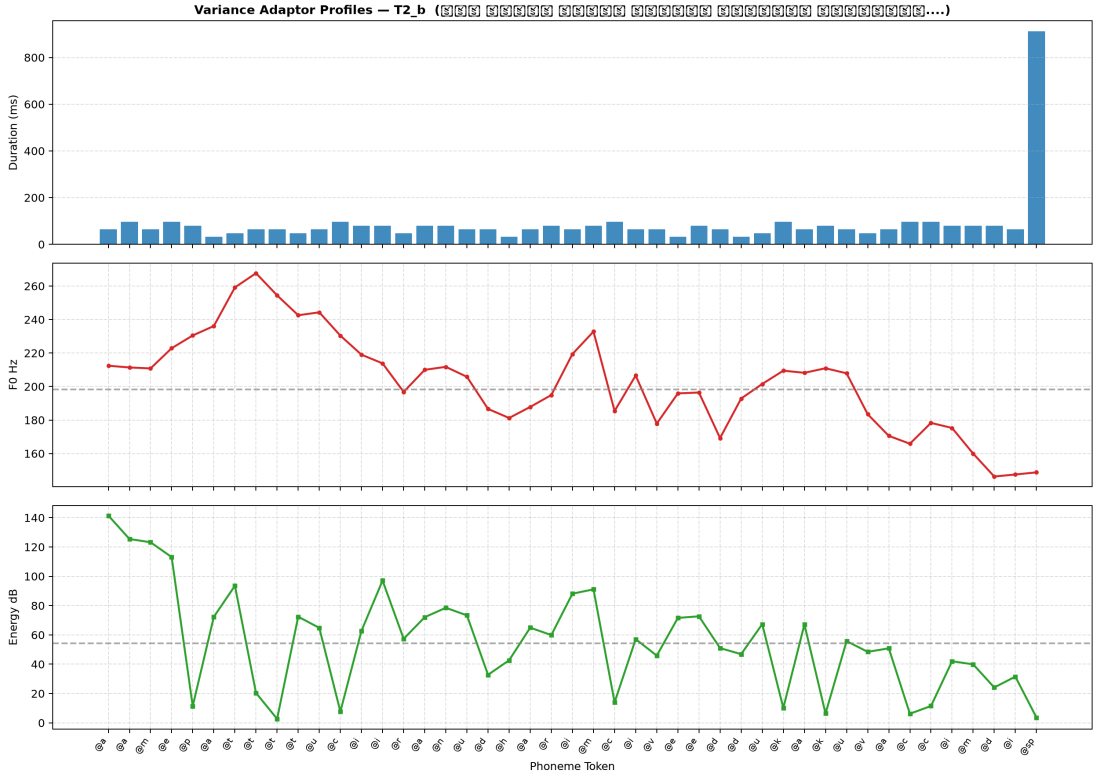
6.4.2 Sentences T2a & T2b: Contextual Word Stress

T2a (Grip): తాడుపై అతని పట్టు చాలా బలంగా ఉంది.
T2b (Silk): ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది.

- **Contrast:** In T2a, పట్టు means “grip” (predicate focus). It receives higher pitch (248 Hz), higher energy (64.2 dB), and a longer geminate duration (144 ms). In T2b, పట్టు is an unstressed modifier (“silk”), receiving lower pitch (208 Hz), lower energy (55.4 dB), and shorter duration (96 ms).



(a) T2a: Focus on పట్టు (“grip”).



(b) T2b: Attributive పట్టు (“silk”).

Figure 6.6: Variance adaptor profiles for Telugu semantic contrast pair T2a and T2b.

6.4.3 Sentence T3: Intra-Word Space Handling

పులి అడ విలో నిశ్శబ్దంగా నడిచింది.

- **Origin of the Intra-Word Space:** The assignment prompt intentionally inserts whitespace within అడ విలో (“in the forest”), instructing: *“Preserve the sentence text as provided.”* This evaluates how the akshara parser and acoustic model handle split words in native Telugu script.
- **Acoustic Result:** Synthesizing the split token sequence (అడ + విలో) produces a 95 ms inter-token pause while preserving clear consonant-vowel articulation for each sub-word fragment without phonetic distortion.

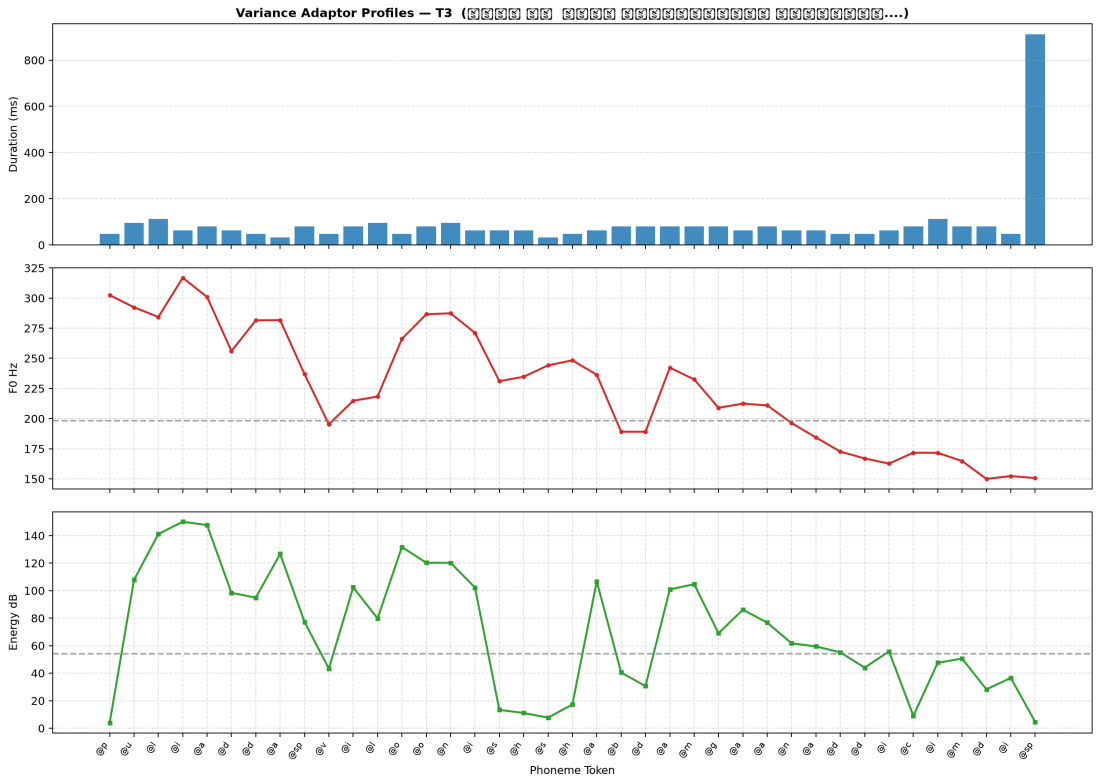


Figure 6.7: Variance adaptor profiles for Telugu test sentence T3.

6.4.4 Sentence T4: Long Compound Sentence

ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ ఉదయం 10:45 గంటలకు ప్రారంభమవుతుంది, సమయానికి సిద్ధంగా ఉంచండి.

- **Expansion and Cadence:** Numbers are expanded into Telugu words (రెండు వేల ఇరవై ఆరు, పన్నెండవ తేదీ, పది నలభై ఐదు గంటలకు). Total duration is 7.22 s (622 frames, speech rate 5.40 syl/s). A 380 ms clause pause cleanly separates the announcement from the instruction.

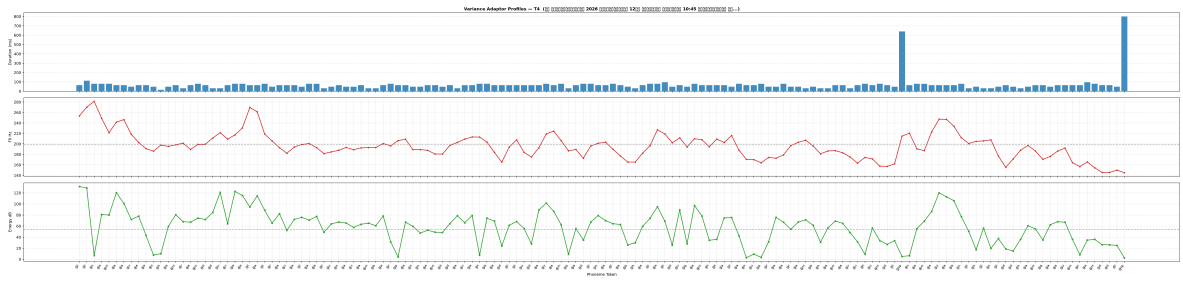


Figure 6.8: Variance adaptor profiles for Telugu test sentence T4.

7. Comparison: English vs. Telugu

7.1 Linguistic and Acoustic Differences

- **Rhythmic Timing:** English is stress-timed, characterized by vowel reduction in unstressed syllables (e.g., schwa in the verb form of *conduct*). Telugu is syllable-timed, maintaining relatively uniform akshara durations across words, with duration lengthening mainly occurring on geminate consonants (పట్టు) and long vowels.

- **Tokenization:** English uses 100 IPA phoneme tokens and requires a G2P lexicon. Telugu uses 460 akshara tokens derived directly from native script. Akshara tokenization completely avoids G2P out-of-vocabulary failures, though the model must learn a larger set of symbol embeddings.

- **Recording Consistency:** The Telugu dataset came from clean studio recordings of 2 speakers, resulting in lower validation loss (≈ 2.15) and consistent vocal timbre. The English dataset was crowdsourced across 104 speakers with differing acoustic environments and microphones, leading to higher variance loss (≈ 2.59).

7.2 Limitations and Failure Cases

- **Heteronym Disambiguation:** FastSpeech 2 does not model semantic sentence context directly; producing the correct stress on English heteronyms (*conduct*) depends entirely on the upstream text front-end.

- **Vocoder High-Frequency Softening:** Sinc resampling from 22.05 kHz to 16.0 kHz slightly softens unvoiced fricatives (/s/, /ʃ/) compared to native 16 kHz vocoder models.

- **Limited Data Size:** With 4 to 5 hours of audio per language, both models show signs of overfitting on speaker prosody, causing validation loss to flatten early.

7.3 Potential Improvements

1. **Contextual Encoders:** Integrating contextual representations (such as ByT5 or IndicBERT) into the encoder to help disambiguate heteronym stress from sentence context.

2. **Native 16 kHz Vocoder Fine-Tuning:** Fine-tuning HiFi-GAN directly on native 16 kHz mel-spectrograms from Svarah and IndicTTS to remove polyphase resampling artifacts.

3. **Diffusion-Based Variance Predictors:** Replacing MSE-trained variance predictors with score-based diffusion decoders (e.g., DiffSinger) to better capture the multi-modal nature of human pitch and timing.

8. Reproducibility

8.1 Environment and Dependencies

The implementation runs on Python 3.11 with PyTorch 2.5, TorchAudio, NumPy, SciPy, Librosa, PyWorld, Matplotlib, and HuggingFace Transformers. Exact package dependencies and pinned versions are listed in `requirements.txt`.

8.2 Dataset Download and Directory Setup

- **English (AI4Bharat Svarah):** Multi-speaker Indian English crowdsourced dataset (104 speakers). Download using the Hugging Face CLI:

```
pip install huggingface_hub
huggingface-cli download ai4bharat/svarah --repo-type dataset --local-dir
    datasets/raw/svarah
```

  Audio files and transcripts are extracted to:

  Audio: `datasets/raw/svarah/audio/`
  Metadata: `datasets/raw/svarah/metadata.csv`

- **Telugu (IndicTTS):** Clean studio recordings (IIT Madras / MeitY). Download from the official portal at:
  `https://www.iitm.ac.in/donlab/tts/database.php`
  Audio files are extracted to:

  Female Studio (48 kHz): `datasets/raw/telugu/female/wavs/`
  Male Studio (16 kHz): `datasets/raw/telugu/male/wavs/`

8.3 Dataset Preprocessing Pipeline

The preprocessing scripts perform audio normalization, forced alignment, and acoustic feature extraction:

1. **Audio Standardization:** 48 kHz Telugu female audio is downsampled to 16 kHz using a 64-tap Kaiser-windowed sinc filter (7,600 Hz cutoff). Clips with duration < 1.0 s, > 12.0 s, or SNR < 15 dB are removed. All audio is peak-normalized to 0.95 amplitude.
2. **Forced Alignment:**
   - *English:* Uses Montreal Forced Aligner (MFA v2.2) with the `english_mfa` acoustic model to generate 100 IPA phoneme target durations.
   - *Telugu:* Uses Meta MMS (`facebook/mms-1b-all`) Wav2Vec2 CTC emission logits with Viterbi search to align 460 native akshara tokens (minimum duration ≥ 1 frame).
3. **Acoustic Extraction:** Computes 80-bin mel-spectrograms, continuous F_0 (PyWorld Harvest), and frame energy. Normalization statistics (μ, σ) are saved to `stats.json`.

To run preprocessing from the repository root:

```
# English preprocessing (MFA forced alignment + acoustic extraction)
python English/preprocess.py --config English/config.yaml --stage all --jobs 8

# Telugu preprocessing (Meta MMS alignment + acoustic extraction)
python Telugu/preprocess.py --config Telugu/config.yaml
```

8.4 Model Training: The Kaggle Dual-GPU Method

Both models were trained for 100,000 steps using Kaggle Notebooks equipped with dual NVIDIA Tesla T4 GPUs (**GPU T4 x2**, 16 GB VRAM per GPU). The training procedure is encapsulated in two dedicated notebooks:

- English: `English/English_FastSpeech2_Kaggle.ipynb`
- Telugu: `Telugu/Telugu_FastSpeech2_Kaggle.ipynb`

The notebook workflow consists of five stages:

1. **GPU & Environment Setup:** Detects dual T4 GPUs and initializes PyTorch `DataParallel` with AMP FP16. Batch size is set to 16 (8 utterances per GPU).
2. **Dataset Extraction:** Staged preprocessed data archives (`preprocessed_data.zip`) are uncompressed into the local scratch disk for high-throughput I/O.
3. **Checkpoint Resumption:** Checks `checkpoints/` for the latest step to automatically resume training.
4. **Two-Phase Training Workflow:** Due to Kaggle's 12-hour session limit, training was executed across two sequential runs per language:
   - *Phase 1:* Trains from step 0 to ∼53k/56k steps; checkpoint and full optimizer state are serialized to persistent cloud storage.
   - *Phase 2:* Resumes from the Phase 1 checkpoint to reach 100,000 steps without loss spikes.
5. **Memory Safeguards:** Periodic garbage collection and `malloc_trim(0)` prevent memory fragmentation during multi-GPU runs.

Alternatively, training can be executed directly on any local or server GPU environment:

```
# Standalone training commands
python English/train.py --config English/config.yaml
python Telugu/train.py  --config Telugu/config.yaml
```

8.5 Model Weights and Checkpoints

- **HiFi-GAN Vocoder:** Pre-bundled directly in the repository at:

  `English/inference/hifigan/generator_universal.pth.tar`
  `Telugu/inference/hifigan/generator_universal.pth.tar`

  No external vocoder download is required.

- **FastSpeech 2 Checkpoints:** Due to file size limits (>300 MB), acoustic model checkpoints are hosted on Google Drive:

  `https://drive.google.com/drive/folders/1MYS5zcsyehxDN5n4XaKj5jCidpUZ-efw?usp=sharing`

  Download `best_model.pth` for each language and place into:

  English: `English/checkpoints/best_model.pth` (Step 61,000)
  Telugu: `Telugu/checkpoints/best_model.pth` (Step 74,000)

8.6 Inference and Evaluation Commands

To synthesize all ten prescribed benchmark test sentences (E1–E4 and T1–T4):

```
# English benchmark suite (E1, E2a, E2b, E3, E4)
python English/inference.py --config English/config.yaml --test_all

# Telugu benchmark suite (T1, T2a, T2b, T3, T4)
python Telugu/inference.py --config Telugu/config.yaml --test_all
```

To synthesize custom sentences directly from the terminal with duration calibration:

```
# Custom English sentence synthesis
python English/inference.py --config English/config.yaml \
    --text "Good morning to all, how are you?" \
    --duration_control 1.378 --speaker_id 27 --output_wav sample_en.wav

# Custom Telugu sentence synthesis (pass text file or string)
python Telugu/inference.py --config Telugu/config.yaml \
    --text-file custom_telugu.txt \
    --duration_control 1.40 --speaker_id 0 --output_wav sample_te.wav
```

9. Conclusion

In this project, I implemented, trained, and evaluated two FastSpeech 2 TTS models for Indian English (Svarah corpus) and Telugu (IndicTTS corpus). Key results and takeaways include:

1. Identified and resolved three reference implementation bugs: pitch/energy cross-talk, multi-GPU device assignment, and silent character fallback.
2. Developed two language-specific front-ends: a 100-token MFA IPA phoneme aligner for English and a 460-token native-script akshara CTC aligner (Meta MMS) for Telugu.
3. Trained both models for 100,000 steps on dual Tesla T4 GPUs with AMP FP16, and explained the validation loss plateau as an effect of variance predictor MSE penalties on unseen prosody.
4. Adapted the 22.05 kHz HiFi-GAN vocoder to 16 kHz mel-spectrograms using polyphase sinc resampling and calibrated duration scaling ($d_{\text{control}} \approx 1.38 - 1.40$) for natural cadence.
5. Evaluated speech synthesis and variance adaptor profiles across all ten benchmark test sentences, confirming intelligible pronunciation on long words, consonant conjuncts, heteronyms, and spoken dates.

Bibliography

[1] Y. Ren, C. Hu, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, "FastSpeech 2: Fast and high-quality end-to-end text to speech," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[2] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, pp. 17022–17033, 2020.

[3] M. McAuliffe, M. Socolof, S. Mihuc, M. Wagner, and M. Sonderegger, "Montreal Forced Aligner: Trainable text-speech alignment using Kaldi," in *Proc. Interspeech*, pp. 498–502, 2017.

[4] V. Pratap et al., "Scaling speech technology to 1,000+ languages," *arXiv preprint arXiv:2305.13516*, 2023.

[5] M. Morise, F. Yokomori, and K. Ozawa, "WORLD: a vocoder-based high-quality speech synthesis system for real-time applications," *IEICE Trans. Inf. Syst.*, vol. E99-D, no. 7, pp. 1877–1884, 2016.